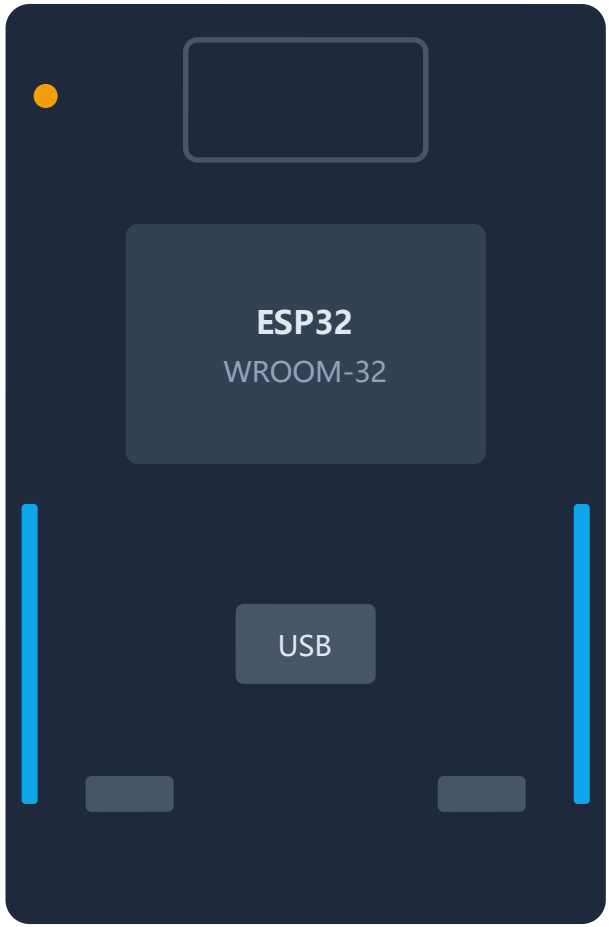
零基础 · 动手做 · 每行代码都解释

ESP32
零基础入门教程



从点亮一颗 LED，到做出能联网的手机可控气象站

12 章 · 18 段完整代码 · 12 次小考 · 1 次大考

作者：老张 · 邮箱：xlgxxc@126.com

配套硬件：ESP32 DevKit V1（约 25 元）· 2026 版教材

ESP32 是什么：25 元，一颗能独立干活的芯片

ONE SENTENCE

把「处理器、内存、WiFi、蓝牙」全塞进一颗指甲盖大的芯片，再焊到一块板子上，25 元卖给你。

它没有屏幕、没有键盘、不会自己说话。

你写程序告诉它「看到温度高于 30 度就去开风扇」，它就老老实实一直做这件事。

240 MHz × 2 核

两个脑子同时算，能一边联网一边刷屏

520 KB 内存

比 30 年前的电脑还多，够放网页和缓存

2 种无线

WiFi 连路由器，蓝牙连手机，都自带

25 个可用引脚

每根线都能接灯、接按钮、接传感器

和谁比	Arduino UNO	ESP32 （本课）	树莓派 Zero 2W
主频	16 MHz 单核	**240 MHz 双核**	1 GHz 四核
内存	2 KB	**520 KB**	512 MB
联网	要外挂模块，另加 15 元	**WiFi + 蓝牙自带**	WiFi 自带，无蓝牙
价格 / 定位	¥60，学电子的老教材	**¥25，联网控制的首选**	¥120，跑 Linux 的小电脑

记住这句 ESP32 **不是电脑的配件**——程序下载进去以后，拔掉电脑它照样自己跑。这就是它和「插在电脑上的传感器」最大的区别。

前世今生：一颗 3 美元的芯片，长成了 10 亿颗

2008

上海张江，一间办公室

张瑞安辞职创业，成立乐鑫。前五年只做 WiFi 芯片，没什么人知道。

2014

ESP8266： 3 美元

一颗芯片同时干「WiFi + 单片机」两份活，价格只有洋货一半。

2016

ESP32： 双核 + 蓝牙

架构从零重做，算力翻几倍。同年开源 ESP-IDF，把工具链全送出来。

2019

科创板首批上市

代码 688018。芯片公司靠创客口碑做到上市，这在当年很罕见。

2020

换成自家的 CPU

ESP32-C3 用自研 RISC-V 内核，不再交授权费，成本又降一截。

2023

累计卖出 10 亿颗

全球 WiFi 芯片出货量连续多年第一。你家路由器、灯泡里可能就有它。

它为什么能赢？不是性能最强，是这三件事同时做到了：

便宜

便宜到学生、老师、小老板都敢买第二块、第三块。一块板子 = 两杯奶茶。

开源

SDK、工具链、原理图全公开。别人家的芯片，连寄存器手册都要签保密协议。

社区

2015 年有人把 Arduino 移植过来，从此「会点灯就会用 ESP32」。

后面还有

2021 年 S3（带 AI 指令）、2023 年 C6（WiFi 6）、2024 年以后 P4 / H4…… 名字越来越花，但**你这块 V1 依然是学得最全的那块**，学完再换型号只是换个名字的事。

三个名字：一颗芯片背后的公司、工程师和社区

你不用记住他们的名字，但要知道：这块 25 元的板子不是天上掉下来的，它背后有具体的公司、具体的人和几十万条代码。

张瑞安

Teo Swee Ann · 乐鑫创始人 / CEO

· 新加坡人，工程师出身，早年在芯片公司 Tensilica 做处理器

· 2008 年在上海张江辞职创业，做出第一颗 ESP 芯片

· 坚持「芯片 + 全开源 SDK」的路线，才把价格和门槛一起打下来

Ivan Grokhotkov

网名 @i_grr · 让 Arduino 跑上 ESP

· 俄罗斯工程师， 2014 年自掏时间写了 ESP8266 的 Arduino 兼容层

· 2015 年 4 月把代码公开，创客圈一夜之间沸腾

· 2016 年加入乐鑫做软件平台负责人——社区方案变成了官方产品

全球社区

arduino-esp32 仓库 · 数千名贡献者

· 有人写库（DHT、OLED、MQTT），有人修 bug，有人翻译文档

· 你遇到的 90% 报错，都有人在 Issue 里问过、也有人答过

· 中文教程、B 站视频、开源项目，是这份课件的事实来源

所以 你今天敲下的每一行 **digitalWrite()**，都是十几年前某个人在深夜里为它写好的。学完这门课，你也可以往下添一行。

重点只有 4 件事，基础只要这么一点点

WHAT YOU WILL LEARN

本课重点：4 件事

01 会接线

看懂引脚图，知道哪根线该插哪儿、为什么必须串电阻

02 会读代码

不背语法，但能逐行说出「这一行在干什么」

03 会调试

灯不亮时不瞎猜，用串口一句一句问它到底怎么了

04 会联网

WiFi → HTTP → MQTT 这条线走通，你的项目就不再是玩具

WHAT YOU NEED

要配合的知识：就这 4 样

- **初中数学**：比例、单位换算（mA / V / Ω / 秒）
- **欧姆定律 U = I × R**：第 3 章会用一整页讲透，现在不会也没关系
- **电脑基本操作**：装软件、解压文件、在设备管理器里找串口
- **认识英文单词**：pinMode 、 delay 这种，会查字典就够

你完全不需要：

不用会 C++ / 不用会画电路板 / 不用学高数 / 不用背引脚顺序（用的时候查表就行）

怎么学

- **看** 先看懂这一页在解决什么问题
- **抄** 代码原样敲一遍，不许复制粘贴
- **改** 改一个数字、换一个引脚，看会发生什么
- **造** 合上课件，做一个自己的小东西

目录

CONTENTS

1 个开篇 + 12 章 · 7 张接线图

18 段完整代码 · 12 次小考 · 1 次大考

建议每周 2 章，6 周学完

00	开篇：ESP32 是什么、怎么学	P2–5 约 2h
01	认识你的开发板	P10–15 约 1h
02	装好开发环境	P16–21 约 1.5h
03	第一个程序：点亮 LED	P22–29 约 2h
04	输入：按键与串口调试	P30–36 约 2h
05	模拟量：ADC 与 PWM	P37–43 约 2h
06	传感器与屏幕：I2C	P44–51 约 2.5h
07	联网：WiFi 与 HTTP	P52–57 约 2h
08	做个网页控制灯	P58–62 约 2.5h
09	MQTT：接入物联网平台	P63–66 约 2.5h
10	掉电不丢：存储与配置	P67–69 约 1.5h
11	同时做多件事	P70–73 约 2h
12	综合项目：桌面气象站	P74–80 约 3h

开篇

它能干什么：四个你现在就能做出来的东西

它不聪明，但能把一件事做到底

没有屏幕、键盘、操作系统，跑不了微信和游戏。

它只擅长一件事：**采集数据 → 做判断 → 联网上报**。

物联网里九成的活儿，就是这一件事。

和电脑最不一样的地方

电脑断电就关机，它通上电 0.3 秒就开始干活；电脑程序崩了会弹报错窗口，它崩了只会静静地不动。

所以后面每一章都要教你同一件事：**用串口看它到底在想什么**。

01 手机遥控的灯

手机网页或 App 发一条指令，人在公司也能开关家里的灯，还能看到它现在到底是开是关。

02 会自己上报的温湿度计

DHT11 每 2 秒采一次温湿度，OLED 屏上显示，同时上报到云端，手机上能看一整天的曲线。

03 门磁与人体感应报警

门一开、人一过，传感器电平变化被立刻捕获，1 秒内推送一条消息到手机。

04 小车、机械臂、调光灯

用 PWM 波控制电机转速与灯亮度，读回传感器做闭环，这就是机器人项目的地基。

开篇

型号那么多，为什么只学 ESP32 DevKit V1

主频

240 MHz 双核

可编程 IO

25 个可用引脚

放程序的地方

4 MB 存储

一块板子的价格

25 元左右

型号	核心 / 主频	存储	参考价	一句话定位
ESP32 DevKit V1	双核 240MHz	4MB	¥25	**教程最多、引脚最全，本课就用它**
ESP32-S3 DevKitC	双核 240MHz	8MB	¥60	要做摄像头、AI 识别、USB 键鼠时选
ESP32-C3 SuperMini	单核 160MHz	4MB	¥15	指甲盖大小，引脚少，适合做成品
ESP8266 NodeMCU	单核 80MHz	4MB	¥15	只有 WiFi 没有蓝牙，老教程多，不建议新买

结论 买 V1 的唯一理由不是它最强，而是**你踩的每一个坑，网上都已经有人踩过并写下了答案。**

开篇

整套材料 66 元，一次买齐不返工

材料	规格（照着买）	数量	价格	为什么需要它
ESP32 开发板	DevKit V1， 30 针， CP2102	1	¥25	整套课的主角，认准 30 针版
USB 数据线	Micro-USB，必须能传数据	1	¥5	只能充电的线是新手第一大坑
面包板	400 孔	1	¥8	免焊接，插错拔出来重插就行
杜邦线	公对公、母对母	各 20 根	¥6	连接板子与元件，两种头都要
LED + 电阻	5mm LED； 220Ω、 10kΩ	各 5 个	¥3	220Ω 限流保护 LED， 10kΩ 做上拉
电位器	10kΩ 旋钮式三脚	1	¥2	学 ADC 采集最直观的器件
DHT11	温湿度模块（带小板）	1	¥5	带转接板的最好，省一个上拉电阻
OLED 屏	0.96 寸 I2C， 128×64	1	¥12	认准四针 I2C 版，别买七针 SPI

买板子认三个字

30 针 · CP2102 串口芯片 · Micro-USB 口。三样对不上，前面的教程就白看。

不建议一次买全

先买前四行（约 44 元）跑通第 4 章，确认板子是好的，再补传感器。

合计

66 元

01

CHAPTER 01

认识你的开发板

先弄清每个引脚是什么，再谈编程。

这一章真正要背下来的，只有三条铁律。

一块板子上只有 8 个部件，认全了就不慌



⑦ 绿灯 = 电源 · ⑧ 两侧蓝条 = 排针

① 天线区

蛇形铜箔，别用手捏、别贴金属壳，否则 WiFi 掉线

② ESP32-WROOM-32 模组

真正的大脑：芯片 + 4MB 存储 + 晶振，全封在金属罩里

③ 3.3V 稳压器

把 USB 的 5V 降到 3.3V，全板引脚都用 3.3V

④ EN 键

复位键，等于电脑的重启，程序卡死时按它

⑤ BOOT 键

下载模式键，上传失败时按住它救命

⑥ Micro-USB 口

只干两件事：供电、和电脑传程序

⑦ 板载 LED

接在 GPIO2，上电就该闪，不闪说明板子或供电有问题

⑧ 两排排针

30 个引脚，是它跟外部世界唯一的通道

30 个引脚分 6 类：能随便用的只有一半

类别	引脚（GPIO 号）	能干什么	新手怎么用
① 安全 IO （15 个）	13 14 16 17 18 19 21 22 23 25 26 27 32 33	输入、输出都行，带内部上拉	**首选，随便用**
② 仅输入（4 个）	34 35 36 39	只能读电平 / 读模拟量	接传感器可以，点灯不行
③ 启动特殊脚（5 个）	0 2 5 12 15	上电瞬间的电平决定启动方式	能用，但别接会把它拉低的元件
④ 调试串口（2 个）	1 （TX） 3 （RX）	USB 下载程序、串口打印	别占用，占了就上传不了
⑤ 内部 Flash （6 个）	6 7 8 9 10 11	芯片自己用，对外不可见	**一根线都别接**
⑥ 电源（4 个）	3V3 / GND / VIN / 5V	供电与地线	3V3 给传感器， VIN 接 5V 输入

背下这句口诀

34–39 只进不出 · 6–11 碰了就废 · 0/2/5/12/15 上电要小心

为什么点亮 LED 用 GPIO2

它是安全 IO，而且板子上本来就焊了一颗 LED 在上面，不用接线就能看到效果。

接线底板： 30 个脚，认这 6 个就能开工

接线底板 • ESP32 DevKit V1（30 针）

照着这张图认脚：印在板子上的名字，就是代码里写的名字

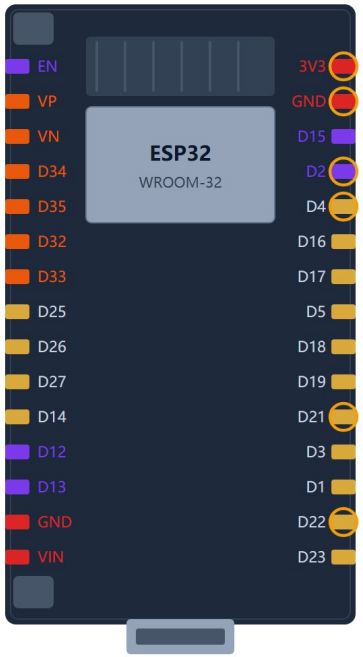
电源：3V3 / VIN / GND
接错会烧板，先读第 13 页的铁律

ADC 输入：34 35 36 39 32 33
只能读电压，不能输出

启动敏感：D2 • D12 • D13 • D15 • EN
上电被拉低会起不来

其它：普通数字 / PWM 引脚
随便用，别超 40mA 就行

本课只用右边这 6 个脚：
3V3 · GND · D2 · D4 · D21 · D22
左边的 D34/D35 到学 ADC 那章才用。



WIRING

怎么用这张图

- 3V3 和 GND： 电源脚，每个模块都要用
- D2 / D4： 本课最常用的两个信号脚
- D21 / D22： I2C 专用，接屏幕用
- 其它脚先别碰，等你看懂电压那一页再说

板子印的名字就是代码里的名字：印 D2 就是 2 号脚，代码里写 2；印 D4 就是 4 号脚。

三条铁律：违反任何一条，板子当场报废

烧掉一块板子，只需要接错一次 5V

下面三条不是 " 注意事项 " ，是硬规则。每次通电之前，花 10 秒对一遍。

01

电压：只能吃 3.3V

GPIO 引脚最高 3.3V 。 5V 的传感器信号直接接上去，芯片当场击穿，没有 " 试试看 " 。要用 5V 器件，中间必须加电平转换模块。

02

电流：每个脚 40mA

一个引脚最多输出 40mA ，全板加起来别超 200mA 。电机、继电器、灯带必须用外部电源供电，板子只负责出 " 信号 " 。

03

手：插拔先断电

带电插线，一瞬间的短路就能烧掉稳压器。养成习惯：拔 USB → 改线 → 再插 USB 。三秒的事，能省 25 块。

4 道题，检验你是不是真看懂了

1. 判断： GPIO34 可以接一颗 LED 并把它点亮。

为什么： 34–39 属于 " 仅输入 " 引脚，内部没有输出电路，接 LED 永远不会亮。

2. 选择：一个 5V 供电的传感器，信号线应该怎么接？

A 直接接 GPIO　　B 经过电平转换模块再接 GPIO　　C 接 3V3 引脚

3. 计算： GPIO2 输出 3.3V，串 220Ω 电阻接红色 LED （压降约 1.9V），电流约多少？

公式： I = (3.3 − 1.9) ÷ 220 ≈ 0.0064A = 6.4mA，在 40mA 上限之内，安全。

4. 填空：上传程序一直失败，除了检查数据线，还可以按住 _____ 键再点上传。

提示：它接在 GPIO0，按住它能让芯片进入 " 下载模式 " 。

参考答案　1. 错（仅输入，不能点灯）　2. B （5V 信号必须降压）　3. 约 6.4mA　4. BOOT

答对 3 题以上再往下走；第 2 题答错的话，回头把 " 三条铁律 " 再看一遍。

02

CHAPTER 02

装好开发环境

总共 20 分钟：装软件、装板卡包、插上线。

90% 的人卡在最后一步的串口驱动上。

三步装好 Arduino IDE，20 分钟能跑第一个程序

① 装 IDE 软件

- 官网 arduino.cc → Software → Windows
- 一路默认选项 " 下一步 " 即可
- 别用微软商店版，串口权限容易出问题

② 装 ESP32 板卡包

- 文件 → 首选项 → 附加开发板管理器网址
- 粘贴下方那行网址，不能有空格
- 工具 → 开发板 → 开发板管理器 → 搜 esp32 → 安装（约 300MB ）

③ 连板子、选端口

- 插上 USB，等 10 秒让驱动装完
- 开发板选 ESP32 Dev Module
- 端口出现 COM3 / COM5 这类字样就算成功

第 ② 步要粘贴的那行网址（只粘这一行）

```
https://espressif.github.io/arduino-esp32/package_esp32_index.json
```

板卡包版本选哪个：开发板管理器里选 **3.x 最新版。**网上很多老教程用的是 2.x，函数写法有差异，本课统一按 3.x 教。

没有 " 端口 " 选项？说明串口驱动没装。看下一页的第 2 号死法。

这 4 个设置项选错，后面全是白忙

设置项（工具菜单里）	选什么	选错的后果
开发板	**ESP32 Dev Module**	引脚映射错乱，程序行为诡异
端口	**COM3 / COM5 …（插上才出现）**	根本点不了上传
Upload Speed	921600（失败就改 115200）	线材差时会超时，降速就能过
Flash Size	4MB (32Mb)	选大了编译不报错，运行直接崩
Partition Scheme	Default 4MB with spiffs	分区不对，OTA 与文件系统都用不了
CPU Frequency	240MHz	跑不满速，WiFi 也可能不稳
PSRAM	Disabled	本板没有 PSRAM，开了会启动失败

还有一处容易被忘

右上角 " 串口监视器 " 的波特率要和代码里 Serial.begin() 一致，本课统一用 **115200**。不一致就是一堆乱码。

选完记得检查一遍

每次换 USB 口、换电脑，端口号都会变，上传失败第一件事就是回来看这里。

点一下 " 上传 " ，芯片里到底发生了什么

1. **写代码**
   在 .ino 文件里敲下程序，先按 Ctrl+S 保存
2. **验证（编译）**
   点左上角 ✓，把 C++ 翻译成芯片能懂的机器码，同时检查语法错误
3. **上传**
   点 →，把编译好的 .bin 通过 USB 送进板子
4. **自动复位进下载模式**
   IDE 通过串口芯片拉低 EN 与 GPIO0 ，让芯片停下来准备接收
5. **烧写并校验**
   一页一页写进 4MB Flash ，写完逐字节比对
6. **自动重启运行**
   芯片重新启动，从 Flash 里取出你的程序，开始执行 setup()

编译成功后，只看这两行

```
Sketch uses 268145 bytes (20%) of program storage space.
Global variables use 21436 bytes (6%) of dynamic memory.
```

第一行是 Flash 占用，第二行是内存占用。任何一个到 100% ，程序就装不下了——那时候该做的是删库、精简，而不是换板子。

看到这行别慌

Connecting....._____ 表示板子正在等下载模式，下面还有一小串点，属于正常握手过程，别急着拔线。

一句话记住：上传 = 把电脑里的 .bin 文件，通过 USB 写进芯片的 Flash ，然后让它重启。

上传失败的 6 种死法，对号入座

屏幕上的现象	真实原因	解法（按顺序试）
Failed to connect to ESP32: Timed out	芯片没进入下载模式	**按住 BOOT → 点上传 → 出现 Connecting 再松开**
端口菜单是灰的 / 一个端口都没有	缺 CH340 或 CP2102 驱动	装驱动，设备管理器里看有没有黄色感叹号
could not open port COMx	串口被别的软件占用	关掉串口监视器、其他 IDE 窗口、串口助手
上传成功，但板子毫无反应	开发板选错 / 引脚号写错	核对 ESP32 Dev Module，核对代码里的 GPIO 号
编译报 was not declared in this scope	少装库，或函数名拼错	按报错行号补 #include；先查拼写
串口里全是乱码、板子反复重启	波特率不对 / 供电不足	监视器调 115200；换 USB 口或换数据线

先怀疑线，再怀疑板

十次上传失败里有六次是 " 只能充电的 USB 线 " 。找一根确定能传文件的线，能省一晚上。

万能复位组合

按住 BOOT 不放 → 点一下 EN 松开 → 再松开 BOOT 。板子强制进入下载模式，几乎能救回所有 " 砖头 " 。

环境装完了？先过这 4 关

1. 选择：上传时提示 "Failed to connect to ESP32: Timed out"，第一步该做什么？

A 换一块新板子　　B 按住 BOOT 键再点上传　　C 重装 Arduino IDE

2. 判断：第一次把端口选成 COM3，以后换 USB 口都不用再选。

为什么：端口号是电脑给每次插入分配的，换口、换线、重启都可能变成 COM4 / COM5 。

3. 填空：代码里写了 Serial.begin(115200)，串口监视器右下角的波特率要选 _____ 。

提示：两边不一致，屏幕上就是一堆看不懂的方块和问号。

4. 简答：编译信息提示 program storage space 已经 100%，你会怎么处理？

思路：先删掉用不到的库和调试代码；仍然不够就换更大的分区方案；最后才考虑换 S3 （8MB）。

参考答案　1. B　2. 错（端口会变，每次都要确认）　3. 115200　4. 删库精简 → 换分区 → 换板

全对：环境这关可以放心了。第 1 题答错：把第 20 页的表格抄一遍贴在桌子前。

03

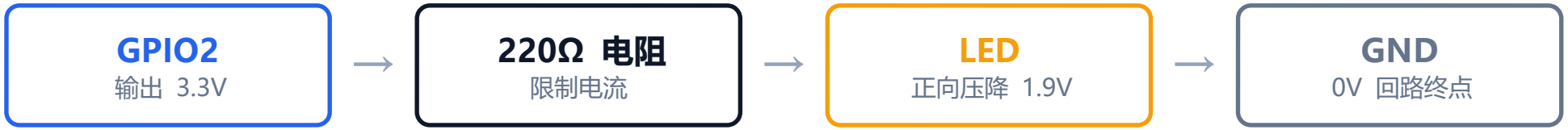
CHAPTER 03

第一个程序：点亮 LED

不接线也能做：板子上本来就焊了一颗灯。

这一章要弄懂的是 setup 、 loop 、电平三个词。

外接 LED 时，为什么必须串一个电阻



不串电阻会怎样

LED 导通后内阻只有几十欧，3.3V 直接加上去，瞬间电流可达几百毫安。结果只有两个：**灯珠烧掉**，或者**引脚被拉伤**。

串上 220Ω，电流被压到几毫安，既够亮又安全。电阻放在灯前面还是后面都行——串联电路里，电流处处相等。

手上没有 220Ω？330Ω、470Ω 也能点亮，只是暗一点。100Ω 以下就别用了。

算一遍，心里有数

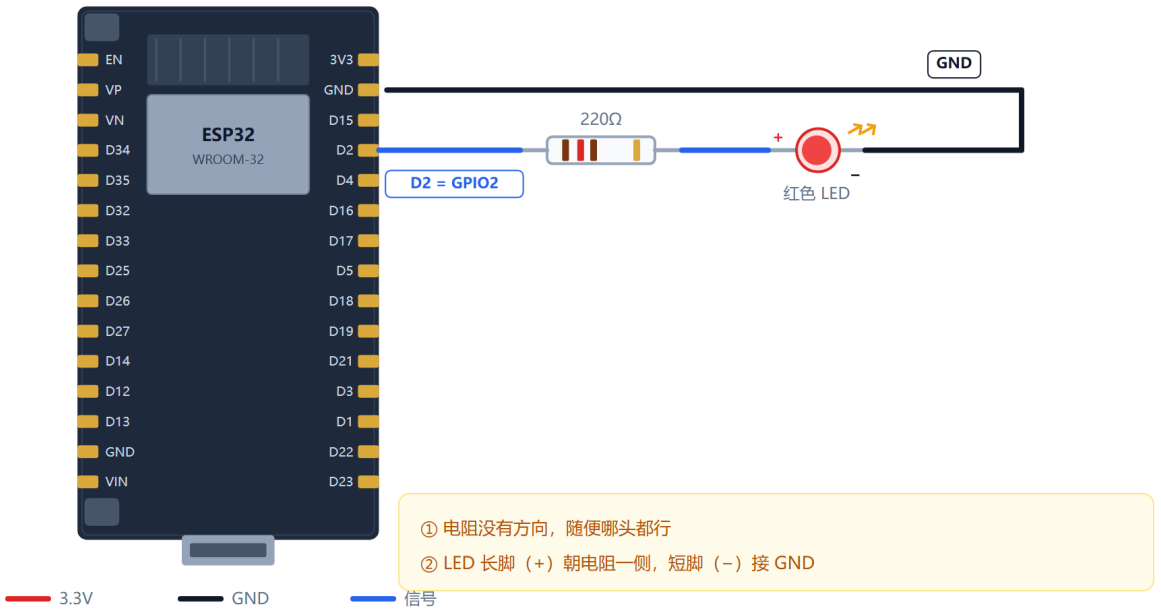
```
I = (U 电源 − U_LED) ÷ R
I = (3.3 − 1.9) ÷ 220
I ≈ 0.0064 A = 6.4 mA
```

6.4mA 远低于 40mA 上限，灯珠也看得清。白色、蓝色 LED 压降约 3.0V，要更亮就用 100Ω。

接线图： D2 → 220Ω → LED → GND

接线图 • 点亮一颗 LED

D2（GPIO2）→ 220Ω → LED 长脚，短脚回 GND



WIRING

三根线，顺序不能反

- D2 → 电阻的任意一端
- 电阻另一端 → LED 长脚（正极）
- LED 短脚（负极）→ GND

电阻必须串上。直接把 LED 接到 3V3 上，几秒钟就烧掉——这一章的标题就是讲这个。

代码 1： Blink，全世界程序员的第一个程序

```
// 例 1：让板载 LED 每秒闪一次
#define LED 2                 // 板载蓝色 LED 接在 GPIO2

void setup() {                // ① 上电后只跑一次
  pinMode(LED, OUTPUT);       // ② 把 GPIO2 设成 " 输出 " 模式
}

void loop() {                 // ③ setup 跑完就进这里，无限循环
  digitalWrite(LED, HIGH);    // ④ 输出 3.3V，灯亮
  delay(1000);                // ⑤ 原地等 1000 毫秒（1 秒）
  digitalWrite(LED, LOW);     // ⑥ 输出 0V，灯灭
  delay(1000);                // ⑦ 再等 1 秒，然后回到 ④
}
```

不用接任何线

开发板上已经焊了一颗蓝色 LED 在 GPIO2，插上 USB 就能看效果。

上传后该看到什么

蓝色小灯亮 1 秒、灭 1 秒，循环不停。不闪就回第 20 页排查。

先动手改一次

把两个 1000 改成 100 再上传。闪得飞快——这就是你在改程序。

逐行拆解：这 7 行代码，每行在干什么

这一行	它在做什么	为什么必须这样写
`#define LED 2`	给数字 2 起个名字，叫 LED	以后换引脚只改这一行；满篇写 2，改起来必漏
`void setup()`	上电只执行一次的初始化函数	芯片每次复位都先跑它，跑完才进 loop
`pinMode(LED, OUTPUT)`	把 GPIO2 设置成 " 输出 " 模式	不设置它只是个悬空引脚，推不动 LED
`void loop()`	setup 跑完后无限重复的函数	单片机不能 " 结束 "，停下来就等于死机
`digitalWrite(LED, HIGH)`	让 GPIO2 输出高电平 3.3V	有了电压差，电流才会流过 LED
`delay(1000)`	让程序原地等 1000 毫秒	不等的话人眼只看到一片亮；但它会卡住整个程序
`digitalWrite(LED, LOW)`	输出低电平 0V	两端没有电压差，灯就灭了

记一个词　HIGH = 高电平 = 3.3V，LOW = 低电平 = 0V。GND 是 0V 的参照基准，所有电压都是相对它说的。

四个 " 你以为 " ，正是新人卡住的真正原因

你可能以为	实际是这样
✗ “ loop() 里的代码只跑一次”	✓ 它是死循环。 setup 一结束就进来，然后无限重复，一秒能跑几十万遍。所以里面不能放 " 只想执行一次 " 的初始化。
✗ “ delay(1000) 是定时器，时间到了再去干别的”	✓ delay 是 " 原地堵车 " ： CPU 就在那儿数数，这 1 秒里读不了传感器、收不了网络数据。要做多件事，就得用 millis() （第 11 章）。
✗ “引脚号可以写成 D2 、 D4”	✓ ESP32 用 GPIO 编号。写 2 就是 GPIO2 ；写 D2 编译器直接报错——那是 NodeMCU 8266 的写法，别混用。
✗ “ HIGH 就是通电， LOW 就是断电”	✓ 它们是 3.3V 和 0V 。很多模块是 " 低电平触发 " ： 给 LOW 才是打开。接手别人的板子，先看它高电平有效还是低电平有效。

判断一个人是不是真入门，就看他知不知道 loop 会无限跑、 delay 会阻塞。这两条错，后面全是玄学。

代码 2：用函数和 for 循环打出 SOS

```
#define LED 2

// 自己造一个动作：亮 onMs 毫秒，灭 offMs 毫秒
void blinkOnce(int onMs, int offMs) {
  digitalWrite(LED, HIGH);
  delay(onMs);
  digitalWrite(LED, LOW);
  delay(offMs);
}

void setup() {
  pinMode(LED, OUTPUT);
}

void loop() {
  for (int i = 0; i < 3; i++) blinkOnce(150, 150);  // 三短 S
  for (int i = 0; i < 3; i++) blinkOnce(600, 150);  // 三长 O
  for (int i = 0; i < 3; i++) blinkOnce(150, 150);  // 三短 S
  delay(2000);                                      // 停 2 秒再来
}
```

三个新语法，一次讲透

`void blinkOnce(int onMs, int offMs)`

自己造一个函数。 **void** 表示不返回结果；括号里是 " 参数 " ，调用时把数值传进去。

`for (int i = 0; i < 3; i++)`

循环 3 次：i 从 0 开始，每圈加 1 ，i 到 3 就停。三短、三长、三短，正是摩斯码 SOS 。

为什么抽成函数

同一个动作要重复 9 次。写成函数只写一遍，改时长只改一个地方——这就是 " 不重复自己 " 。

灯会闪了，再来回答这 4 个问题

1. 选择： loop() 里的代码，执行顺序是?

A 只执行一次　　B setup 结束后无限循环执行　　C 和 setup 同时执行

2. 判断： GPIO2 直接接一颗 LED 到 GND，中间不串电阻也没关系。

为什么： LED 内阻极小，直接接 3.3V 电流会冲到几百 mA，灯珠和引脚都可能烧掉。

3. 计算：红色 LED 压降 1.9V，串 470Ω 电阻接到 3.3V，电流约多少?

公式： I = (3.3 − 1.9) ÷ 470 ≈ 0.003A = 3mA，能亮，但偏暗。

4. 改错：下面的代码为什么灯不闪?

```
void setup(){ pinMode(2, INPUT); }    void loop(){ digitalWrite(2, HIGH); delay(500); }
```

参考答案　1. B　2. 错（必须串电阻）　3. 约 3mA　4. 两处错： pinMode 该写 OUTPUT；循环里只有 HIGH 没有 LOW，灯一直亮

第 4 题是本章最关键的一题——能一眼看出这两处错，说明你已经会读代码了。

04

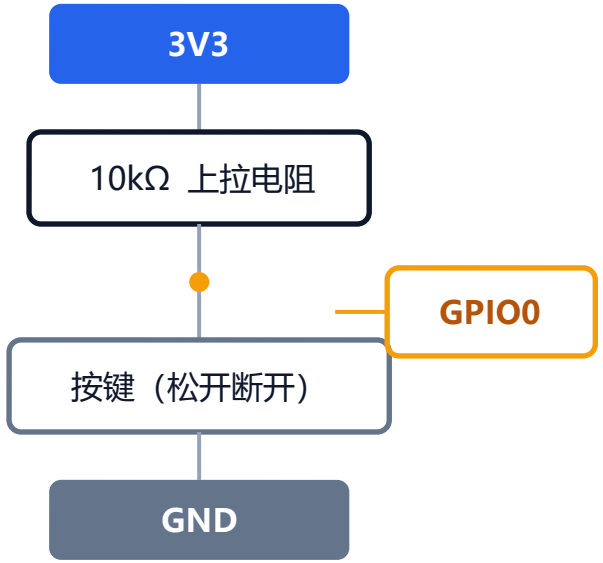
CHAPTER 04

输入：按键与串口调试

让板子 " 听见 " 你按了一下，也让你 " 看见 " 它在想什么。

这一章的串口，是全篇最好用的排错工具。

引脚一 " 悬空 "，读出来的就是一堆随机数



松开时靠电阻拉到 3.3V，按下时被直接拉到 0V

为什么必须给个 " 默认值 "

按键松开时，引脚跟任何东西都没连，这叫**浮空**。浮空的引脚像一根小天线，会拾取周围的电磁噪声，读出来忽 0 忽 1—— 程序就开始莫名其妙地乱闪。

解决办法是给它一个明确的默认电平：用 10kΩ 电阻稳稳拉到 3.3V （上拉）或 0V （下拉）。电阻够大，按键一按就能轻松把电平拉过去。

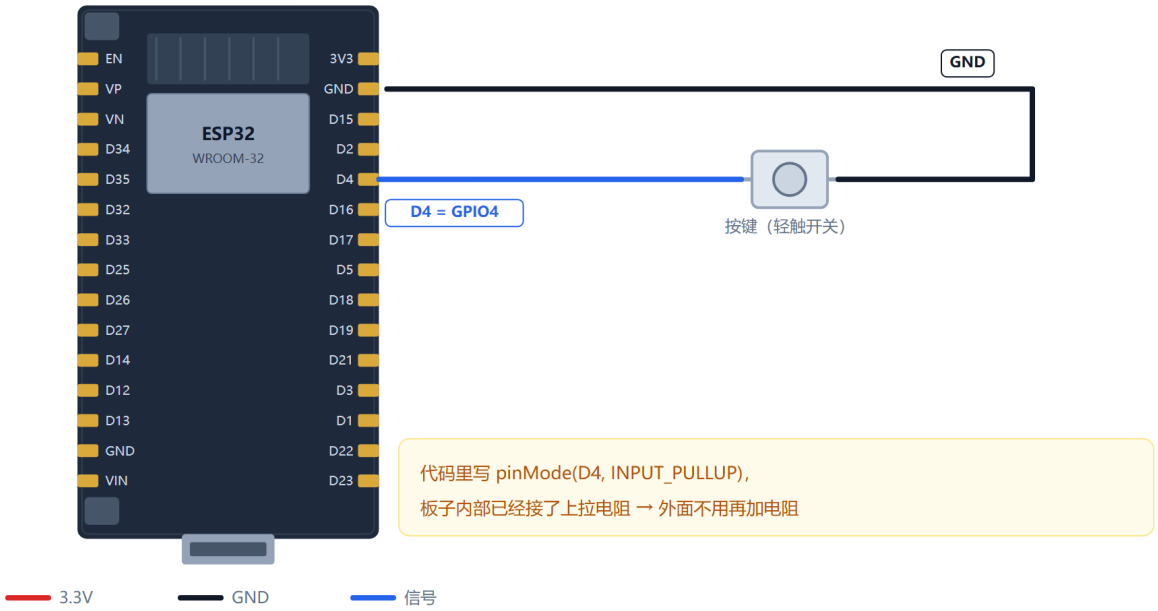
> ESP32 内部自带约 45kΩ 上拉电阻。所以只要写 **pinMode(0, INPUT_PULLUP)**，一个外部电阻都不用加。

接法固定成两句：**按键一端接 GPIO0，另一端接 GND**。板载的 BOOT 键本来就是这么接的，所以第 4 章不用插一根线。

接线图： D4 接按键，另一头回 GND

接线图 · 按键开关灯

D4（GPIO4）接按键一头、另一头接 GND；LED 电路不用动，还是 D2



WIRING

只要两根线

- D4 → 按键的一只脚
- 按键另一只脚 → GND
- LED 电路保持不动，还是接在 D2 上

代码里写 INPUT_PULLUP，上拉电阻在芯片内部，外面不用再加。按下读到 0，松开读到 1。

代码 3：按一下板载 BOOT 键，灯的状态翻转

```
#define BTN 0                // 板载 BOOT 键就是 GPIO0
#define LED 2
bool lastState = HIGH;       // 记住上一次的电平

void setup() {
  Serial.begin(115200);      // 打开串口，波特率 115200
  pinMode(BTN, INPUT_PULLUP);  // 输入 + 打开内部上拉
  pinMode(LED, OUTPUT);
}

void loop() {
  bool now = digitalRead(BTN);          // 读当前电平
  if (lastState == HIGH && now == LOW) {  // 只在 " 按下那一瞬间 " 成立
    digitalWrite(LED, !digitalRead(LED)); // 取反：亮变灭，灭变亮
    Serial.println(" 按键被按下 ");
  }
  lastState = now;    // 存下来，下一圈用来比较
  delay(20);          // 简单消抖，等抖动过去
}
```

`digitalRead(BTN)`

读回来的只有两种值： HIGH 或 LOW 。

`lastState == HIGH && now == LOW`

&& 是 " 并且 " 。上一次是高、这一次是低，才说明刚刚按下。

为什么非要判断 `lastState`

不判断的话，手指按住的那 1 秒里程序已经循环了几十次，灯会疯狂乱闪。

`!digitalRead(LED)`

读出来取反再写回去。一行代替 if / else 两行，是 C 语言里最常见的写法。

`delay(20)`

机械按键按下瞬间会抖动 5–20ms ，产生一串假信号。等 20ms 再读就干净了。

按住 BOOT 再松开 USB ？那是下载模式。正常按一下就好。

串口：板子没有屏幕，这是它唯一的嘴

6 个函数，覆盖 95% 的调试场景

`Serial.begin(115200)`	打开串口，只能调一次，放在 setup 里
`Serial.print(x)`	打印，不换行。拼接一句话时用
`Serial.println(x)`	打印并换行。每条独立信息都用它
`Serial.printf(fmt, ...)`	格式化打印，一次输出多个变量最省事
`Serial.available()`	电脑发来了几个字节？ 0 表示没收到
`Serial.read()`	读走一个字节。做 " 电脑发指令 " 时用

打开方式：右上角放大镜图标 → 选 115200 → 按一下板子的 EN 键看输出。

printf ： 一行输出多个变量

```
int n = 0;

void setup() {
  Serial.begin(115200);
}

void loop() {
  n++;
  Serial.print(" 计数 : ");
  Serial.println(n);
  Serial.printf("n=%d, 毫伏 =%d\n", n, n * 3);
  delay(1000);
}
```

%d 是整数占位符， **%f** 是小数，

是换行。后面的变量按顺序填进去。

代码 4：不用 delay 的按键消抖

```
#define BTN 0
#define LED 2
bool lastRead = HIGH;     // 上一次读到的原始电平
bool stable   = HIGH;     // 确认稳定后的状态
unsigned long tEdge = 0;  // 电平最后一次变化的时刻
void setup() {
  pinMode(BTN, INPUT_PULLUP);
  pinMode(LED, OUTPUT);
}
void loop() {
  bool now = digitalRead(BTN);
  if (now != lastRead) {   // 第 1 步：电平一变，记下时刻
    tEdge = millis();
    lastRead = now;
  }
  if (millis() - tEdge > 30 && now != stable) {
    stable = now;          // 第 2 步：稳定 30ms 才承认
    if (stable == LOW) {   // 第 3 步：确认是 " 按下 "
      digitalWrite(LED, !digitalRead(LED));
    }
  }
}
```

三步，把抖动挡在门外

1 记时刻

读到电平跟上次不一样，就用 millis() 记下当前毫秒数。

2 等稳定

从那一刻起超过 30ms 没有再变，说明抖动结束了。

3 再确认

只有稳定后的新状态和已确认状态不同，才算真按了一下。

为什么不用 delay(20)？因为 delay 期间程序完全停住。一旦你同时要读传感器、要刷新屏幕，delay 就会让所有事情一起卡住。

millis() 返回开机至今的毫秒数，约 49 天会归零一次。用减法比较（now − t），归零也不会出错。

板子能 " 听见 " 你了，回答这 4 题

1. 选择：按键一端接 GPIO0 、另一端接 GND ，代码里该写哪种模式?

A pinMode(0, INPUT)　　B pinMode(0, INPUT_PULLUP)　　C pinMode(0, OUTPUT)

2. 判断：引脚不接任何东西时， digitalRead 会稳定返回 LOW 。

为什么：浮空引脚会随环境噪声乱跳，读到的值完全不可信，必须靠上拉 / 下拉给个默认电平。

3. 解释：代码里为什么必须有 lastState 这个变量?

思路： loop 每秒跑几万次，不比较 " 上一次 " ，按住的一瞬间会被重复触发几十次。

4. 填空：串口监视器一片乱码，最可能的原因是波特率和代码里的 _____ 不一致。

提示：本课统一 115200 。另外 USB 线接触不良也会乱码。

参考答案　1. B　2. 错（会乱跳）　3. 用来只在 " 按下瞬间 " 触发一次　4. Serial.begin()

第 3 题能讲清楚，说明你已经理解 " 事件 " 和 " 状态 " 的区别——这是后面所有联网项目的基础。

05

CHAPTER 05

模拟量： ADC 与 PWM

前面只会读 " 有电 / 没电 " ，这一章开始读 " 多少伏 " 。

反过来，也能输出 " 半亮 " 而不是只有全亮全灭。

ADC：把 0~3.3V 切成 4096 份



0V	0.8V	1.65V	2.5V	3.3V
读数 0	读数 1024	读数 2048	读数 3103	读数 4095

分辨率

12 位 = 4096 档

2 的 12 次方是 4096。每一档大约 0.8 毫伏，电压的细微变化也能分辨。

读一次模拟量，只做两件事

① 告诉芯片 " 用几位的精度 "——analogReadResolution(12)；

② 读引脚—— analogRead(34)，拿回一个 0–4095 的整数。

想直接要电压值，就用 analogReadMilliVolts(34)，它已经按芯片出厂校准曲线校正过，比你自己拿 3.3 去乘更准。

能用 ADC 的引脚只有 GPIO32–39 （ADC1）。这 8 个脚开了 WiFi 也照样能读，原因见下一页。

ADC 的 4 个坑，全都栽过一遍

坑	现象	怎么办
① ADC2 与 WiFi 抢资源	一开 WiFi，analogRead(2) 就返回 0 或乱跳	**只用 GPIO32–39（ADC1 组）**
② 两端不准	接近 0V 和 3.3V 时，读数能差几十毫伏	能用就行；要精度就外接 ADS1115 模块
③ 输入不能超 3.3V	5V 信号进来：读数满格，长期会损伤引脚	两个电阻分压，或直接换 3.3V 传感器
④ 34–39 只能输入	没有内部上拉，也不能输出	信号线接它；别指望它供电或点灯

读数跳动怎么办

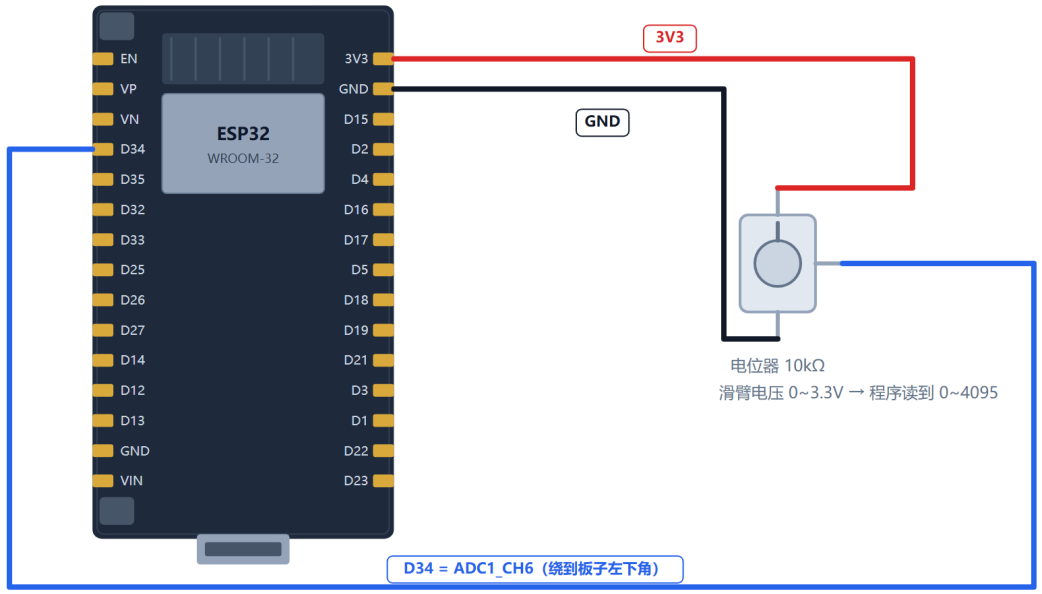
连读 10 次取平均，是最省事的滤波。还跳就在信号线与 GND 之间并一个 0.1μF 电容，把高频噪声直接导走。

一句话：ADC1 读值，ADC2 别碰。**只要项目里要联网，就锁定 GPIO32/33/34/35/36/39。**

接线图：电位器两端接电源，中间接 D34

接线图 • 电位器读电压

两端接 3V3 与 GND，中间滑臂接 D34（ADC1，只能输入不能输出）



WIRING

电位器就是三个脚的可变电阻

- 一端 → 3V3
- 另一端 → GND
- 中间滑臂 → D34 （ADC1 通道 6）

D34 只能输入，不能输出。ADC2 的脚（D2、D4 这些）在连着 WiFi 时读不到数，所以这里必须用 D34。

代码 5：转一下电位器，串口就跟着变

```
#define POT 34    // 电位器中间脚接 GPIO34

void setup() {
  Serial.begin(115200);
  analogReadResolution(12);  // 12 位精度：0~4095
}

void loop() {
  int raw = analogRead(POT);          // 原始读数
  int mv  = analogReadMilliVolts(POT); // 校准后毫伏值
  int pct = map(raw, 0, 4095, 0, 100); // 换算成 0~100

  Serial.printf("raw=%4d mv=%4dmV pct=%3d%%\n", raw, mv,
pct);
  delay(200);  // 每秒读 5 次足够了
}
```

接线只有三根

电位器三个脚：两边分别接 **3V3** 和 **GND**，中间抽头接 **GPIO34**。接反了不要紧，只是旋钮方向反过来。

map() 怎么理解

map(值，原下限，原上限，新下限，新上限）是线性换算：把 0–4095 等比缩到 0–100。

注意它不做限幅：输入超范围，输出也会超范围，别指望它替你挡。

为什么还打 mv

raw 是芯片内部刻度，mv 才是真实电压。做项目时判断阈值（比如 " 低于 1.5V 报警 "），用 mv 才有物理意义。

代码 6：让灯 " 半亮 "，用 PWM 呼吸

```
#define LED 2

void setup() {
  pinMode(LED, OUTPUT);
}

void loop() {
  // 由暗到亮：占空比 0 → 255
  for (int d = 0; d <= 255; d += 5) {
    analogWrite(LED, d);   // 3.x 默认 8 位：0~255
    delay(15);
  }
  // 由亮到暗
  for (int d = 255; d >= 0; d -= 5) {
    analogWrite(LED, d);
    delay(15);
  }
}
```

PWM 不是 " 输出半个 3.3V"

引脚只会输出 3.3V 或 0V 两种状态。PWM 让它每秒开关上千次，**开的时间占多少比例**，就叫占空比。灯看起来就是 " 半亮 " 。



analogWrite 默认频率约 1kHz，人眼完全看不出闪烁。要是驱动电机，频率太低会听到 " 嗡嗡 " 声，那就得用 LEDC 手动设频率。

> 老教程里的 ledcSetup + ledcAttachPin 是 2.x 写法。3.x 已换成 ledcAttach(pin, 频率，位数)，看到别慌。

会读电压、会调亮度，再来这 4 题

1. 计算： analogRead 返回 2048，输入电压大约是多少?

思路： 2048 ÷ 4096 = 一半， 3.3V × 0.5 ≈ 1.65V 。这就是刻度尺中间那一格。

2. 判断：项目要连 WiFi，模拟传感器可以接 GPIO25 。

为什么： GPIO25 属于 ADC2，一旦开启 WiFi 就读不出值。联网项目只能用 GPIO32–39 。

3. 选择： analogWrite(LED, 64)，灯的亮度大约是?

A 全亮　　B 25% 左右　　C 关闭

4. 解释：为什么说 PWM 是在 " 骗眼睛 "，而不是真的输出了 1.65V ?

思路：引脚只有 3.3V 和 0V 两种状态，靠高频开关 + 占空比，让灯的平均功率变小，人眼看到的是 " 变暗 " 。

参考答案　1. 约 1.65V　2. 错（ADC2 不可用）　3. B （64/255 ≈ 25%）　4. 引脚只有通断两态，靠占空比改变平均功率

第 2 题是后面所有联网项目的分水岭，答错一定要回头改引脚。

06

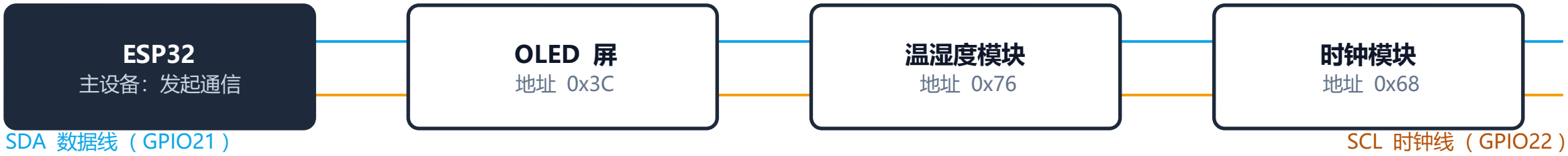
CHAPTER 06

传感器与屏幕： I2C

两根线，就能挂一串设备。

这一章之后，你的板子终于能 " 看见 " 温度和湿度了。

I2C：两根线，挂一串设备，各报各的名字



地址就是门牌号

主设备先喊地址，被喊到的设备才应答。同一型号的模块地址通常固定，所以同一总线挂两个一模一样的 OLED 会打架。

接线只有 4 根，每个模块都一样

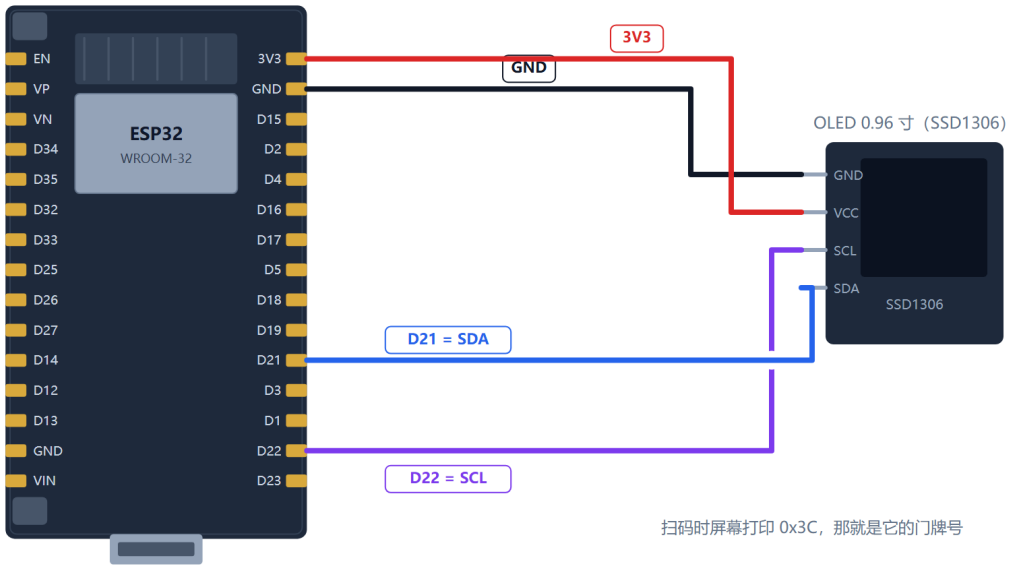
VCC	GND	SDA	SCL
→ 3V3	→ GND	→ GPIO21	→ GPIO22

VCC 一定接 3V3，不要接 VIN/5V。OLED 接 5V 会烧。

接线图： OLED 的四根线（I2C）

接线图 • OLED 屏（I2C）

四根线：GND→GND、VCC→3V3、SCL→D22、SDA→D21；接反了不亮



扫码时屏幕打印 0x3C，那就是它的门牌号

WIRING

I2C 就是两根数据线

- GND → GND
- VCC → 3V3
- SCL → D22
- SDA → D21

SDA / SCL 接反，屏幕不亮也不报错。扫描时一个地址都扫不到，就回来把这两根线换一下。

代码 7：先扫一遍，看看到底插了几个设备

```
#include <Wire.h>

void setup() {
  Serial.begin(115200);
  Wire.begin(21, 22);      // SDA=21, SCL=22，只调一次
  Serial.println("开始扫描 I2C 总线...");
}

void loop() {
  int found = 0;
  for (uint8_t addr = 1; addr < 127; addr++) {
    Wire.beginTransmission(addr);      // 敲这个门牌号
    if (Wire.endTransmission() == 0) {  // 返回 0 = 有人应
      Serial.printf("发现设备: 0x%02X\n", addr);
      found++;
    }
  }
  if (found == 0) Serial.println("一个都没找到，检查接线");
  delay(3000);
}
```

这三行是 I2C 的全部套路

Wire.begin(21, 22) 打开总线； beginTransmission(addr) 敲门； endTransmission() 返回 0 表示有人应。

常见地址，记几个够用

0x3C	0.96 寸 OLED （SSD1306）
0x76 / 0x77	BME280 温湿压传感器
0x68	DS3231 时钟模块
0x27 / 0x3F	PCF8574 转 IO 板 / LCD 背板

扫不到设备：先查 VCC 有没有接 3V3，再查 SDA/SCL 是不是接反了。

代码 8：读温湿度，第一次让板子感知世界

```
#include <DHT.h>              // 需要先装库

#define DHTPIN  4
#define DHTTYPE DHT11         // 换 DHT22 只改这一行

DHT dht(DHTPIN, DHTTYPE);

void setup() {
  Serial.begin(115200);
  dht.begin();                // 启动传感器
}

void loop() {
  float t = dht.readTemperature();  // 摄氏度
  float h = dht.readHumidity();     // 百分比

  if (isnan(t) || isnan(h)) {       // 读失败会是 NaN
    Serial.println(" 读取失败，检查接线 ");
    return;                         // 这一圈直接结束
  }
  Serial.printf(" 温度 %.1f C   湿度 %.1f %%\n", t, h);
  delay(2000);                      // DHT11 至少间隔 1 秒
}
```

三步就能跑起来

① 库管理器装 **DHT sensor library** （Adafruit）；② 接线：VCC→3V3，GND→GND，DATA→GPIO4；③ 上传，打开串口。

三个细节决定成败

isnan() 是 " 是不是无效数字 " 。读失败时函数返回 NaN，不判断就会打印出一堆 nan 。

return 让这一轮 loop 立刻结束，避免用错误数据继续算。

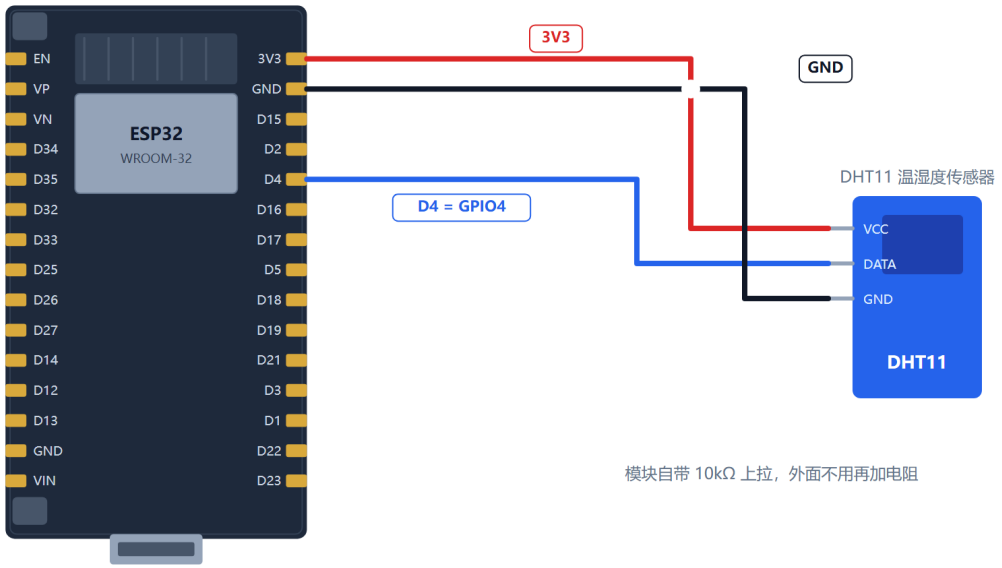
delay(2000) 不是随便写的：DHT11 采样太频繁会返回错误值。

如果是四脚裸传感器（不是模块），DATA 与 VCC 之间要加一个 4.7k–10k 上拉电阻。

接线图： DHT11 的三根线

接线图 • 读温湿度 DHT11

三根线：VCC→3V3、DATA→D4、GND→GND；接反了会烧传感器



模块自带 10kΩ 上拉，外面不用再加电阻

WIRING

模块三个脚，别插反

- VCC → 3V3
- DATA → D4
- GND → GND

VCC 和 GND 接反会烧模块。模块自带 10kΩ 上拉电阻，外面不用再加。

代码 9： 0.96 寸 OLED，让数据看得见

```
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
// 宽 128，高 64，用 Wire，复位脚 -1 （无）
Adafruit_SSD1306 oled(128, 64, &Wire, -1);
void setup() {
  Serial.begin(115200);
  Wire.begin(21, 22);
  if (!oled.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
    Serial.println("OLED 初始化失败，查地址 ");
    for (;;);            // 停在这里方便排查
  }
  oled.clearDisplay();    // 清屏
  oled.setTextColor(SSD1306_WHITE);
  oled.setTextSize(2);    // 字放大 2 倍
  oled.setCursor(0, 0);   // 从左上角开始写
  oled.println("Hello");
  oled.println("ESP32");
  oled.display();         // 不调用它，屏幕永远黑的
}
void loop() {}
```

库装两个，缺一不可

Adafruit SSD1306 负责显示屏， **Adafruit GFX** 负责画图写字。装前者时会提示装后者，点 " 全部安装 " 。

屏幕坐标系：左上角是（0,0）



x 向右增， y 向下增。

setCursor(x, y) 决定下一句从哪儿开始写。

屏幕是双缓冲的：先往芯片内存里画， **display()** 才真正刷到屏上。忘了它，屏幕上永远是上一帧。

会说话了，会显示了，来答题

1. 选择： I2C 总线上， ESP32 的默认 SDA 与 SCL 分别是哪两个引脚?

A GPIO21 / GPIO22　　B GPIO22 / GPIO23　　C 任意两个引脚都行，不用指定

2. 判断：同一台 ESP32 可以同时接一块 OLED （0x3C）和一块 BME280 （0x76）。

为什么：地址不同就能共存。地址冲突才是问题，比如两块 0x3C 的 OLED 就挂不上去。

3. 填空： OLED 上写满了内容，但屏幕一直是黑的，最可能是漏了 _____ 。

提示：屏幕是双缓冲的，画完必须刷一次才显示。

4. 解释： DHT11 读出来是 nan，你会按什么顺序排查?

顺序：① 接线（VCC 是否 3V3 、 DATA 是否 GPIO4 ）② 是否有上拉电阻 ③ 读取间隔是否小于 1 秒 ④ 换一个 DATA 引脚再试

参考答案　1. A　2. 对　3. oled.display()　4. 接线 → 上拉 → 间隔 → 换脚

第 4 题的排查顺序，就是以后所有 " 读不到数据 " 的通用套路：先硬件、再时序、最后软件。

07

CHAPTER 07

联网：WiFi 与 HTTP

从这里开始，你的板子接上了整个互联网。

代价是：多了一个 2.4G 的坑，和多一堆状态要判断。

WiFi 的三种身份：连别人、被连、两个都干

STA

像手机一样去连路由器

ESP32

路由器

互联网

最常用。连上之后路由器给它一个 IP，它就能访问网络、上传数据。代价是：必须知道 WiFi 名称和密码。

AP

自己开一个热点

手机

ESP32 热点

板子变成路由器，手机直接连它。没有互联网，但现场调试、无网环境控制特别方便。

STA+AP

两个模式同时开

路由器 ↔ ESP32 ↔ 手机

既能上云，又能本地直连。代价是更耗电、更占内存，代码复杂度也上去了。

最常踩的坑

ESP32 只支持 **2.4GHz** WiFi，不支持 5GHz。很多人家里的路由器只开了 5G，代码没错却一直连不上——去路由器后台把 2.4G 打开。

代码 10：连上路由器，拿到自己的 IP

```
#include <WiFi.h>
const char* SSID = " 家里的 WiFi 名 ";
const char* PASS = "WiFi 密码 ";
void setup() {
  Serial.begin(115200);
  WiFi.mode(WIFI_STA);      // 手机模式：去连路由器
  WiFi.begin(SSID, PASS);
  Serial.print(" 正在连接 ");
  unsigned long t0 = millis();
  while (WiFi.status() != WL_CONNECTED && millis() - t0 < 15000) {
    delay(500);
    Serial.print(".");
  }
  if (WiFi.status() == WL_CONNECTED) {
    Serial.printf("\nIP = %s\n", WiFi.localIP().toString().c_str());
  } else {
    Serial.println("\n 失败：查密码，确认是 2.4G");
  }
}
void loop() {}
```

四行关键代码，逐个说清

`WiFi.mode(WIFI_STA)`

先定身份再连接。不写也行，默认就是 STA，但写出来更清楚。

`WiFi.begin(SSID, PASS)`

只是 " 发起连接 "，它会立刻返回，不会等连上。

`while (... && millis() - t0 < 15000)`

死等会卡住程序。设 15 秒超时，失败了还能继续走。

`WiFi.localIP().toString().c_str()`

IP 对象 → 字符串 → C 风格字符串，printf 才能打印。三层套娃，少一层都编译不过。

拿到 IP 很重要：同一个局域网里，用手机浏览器访问这个 IP，就能看到板子做的网页（第 8 章）。

代码 11：向一个网址要数据

```
#include <WiFi.h>
#include <HTTPClient.h>
const char* URL = "http://t.weather.itboy.net/api/weather/city/101010100";
void setup() {
  Serial.begin(115200);
  WiFi.begin("SSID", "PASS");     // 换成你自己的
  while (WiFi.status() != WL_CONNECTED) delay(500);
  Serial.println(WiFi.localIP());
}
void loop() {
  if (WiFi.status() != WL_CONNECTED) return;
  HTTPClient http;                // 造一个请求对象
  http.begin(URL);                // 去哪里
  int code = http.GET();          // 发出去，拿状态码
  if (code == 200) {              // 200 = 成功
    Serial.println(http.getString());
  } else {
    Serial.printf("失败 code=%d\n", code);
  }
  http.end();                     // 必须释放
  delay(60000);                   // 一分钟一次
}
```

一次 HTTP 请求的四件事

URL	去哪儿要：协议 + 域名 + 路径
方法	GET 是 " 取 "， POST 是 " 交 "。拉数据用 GET
状态码	200 成功；404 路径错；401 没权限
正文	返回的内容，通常是 JSON 文本

注意 URL 开头是 **http://** 不是 https。ESP32 访问 https 需要证书，新手先别碰。

这个天气接口是第三方免费服务，可能某天失效。失效了换个返回 JSON 的接口，代码结构完全不用改。

代码 12：从一大坨文字里，只取你要的两个字

返回的内容长这样（简化版）

```
{
  "cityInfo": { "city": "北京市" },
  "data": {
     "wendu": "18",
     "ganmao": "天气转凉，注意保暖"
  }
}
```

像俄罗斯套娃：外层叫 cityInfo，里层叫 city。取值就是一层一层往下钻：**doc["data"]["wendu"]**。

解析代码，只要 8 行

```
#include <ArduinoJson.h>

void parseWeather(String body) {
  JsonDocument doc;     // v7 写法，不用写容量
  if (deserializeJson(doc, body)) {
    Serial.println("JSON 解析失败");
    return;
  }
  const char* city = doc["cityInfo"]["city"];
  const char* wd   = doc["data"]["wendu"];
  Serial.printf("%s 现在 %s 度\n", city, wd);
}
```

deserializeJson 返回 0 表示成功。返回值非 0 就直接打印 "失败" 并结束，不要拿坏数据往下算。

库里搜 ArduinoJson 装最新版（7.x）。6.x 老教程写 DynamicJsonDocument doc(1024)，7.x 已简化成 JsonDocument doc。

板子已经上网了，来答这 4 题

1. 判断：代码完全正确，但一直连不上家里的 WiFi，最可能是什么原因？

答：路由器只开了 5GHz。ESP32 只认 2.4GHz，去路由器后台把 2.4G 频段打开。

2. 选择：WiFi.begin() 调用之后，程序会怎样？

A 一直等到连上才往下走　　B 立刻返回，需要自己查 WiFi.status()　　C 自动重启

3. 填空：HTTP 返回状态码 _____ 表示请求成功。

提示：404 是路径写错，401 是没权限。

4. 解释：JSON 里 data 下面有个 wendu，代码怎么写才能取到它？

答：doc["data"]["wendu"]，一层一层往下钻；取到的可能是字符串，要转数字用 doc["data"]["wendu"].as<int>()。

参考答案　1. 只开了 5GHz（或密码错）　2. B　3. 200　4. doc["data"]["wendu"]

第 2 题答错的话，重看第 54 页的 while 循环——那是所有联网项目的心跳。

08

CHAPTER 08

做个网页控制灯

不用装 App，不用连云端。

板子自己就是一个网站，手机浏览器打开就能用。

原理：板子当网站，手机当浏览器



① 浏览器发请求

你在地址栏输入 IP，或点页面上的 " 开灯 " 链接，浏览器就发出一条 GET 请求。

② 板子执行动作

WebServer 收到 /on 这个路径，就去调用你注册的 handleOn 函数，把引脚拉高。

③ 返回一段内容

处理完必须 " 回话 "，否则浏览器一直转圈。send(200, 类型，内容）就是回话。

手机必须和 ESP32 在同一个 WiFi 下。用 4G 流量是打不开的——那是 " 远程控制 "，要用第 9 章的 MQTT。

代码 13 （上）：先注册路由，再等客人上门

```
#include <WiFi.h>
#include <WebServer.h>

WebServer server(80);        // 监听 80 端口（网页默认端口）
const int LED = 2;

void setup() {
  Serial.begin(115200);
  pinMode(LED, OUTPUT);

  WiFi.begin("SSID", "PASS");
  while (WiFi.status() != WL_CONNECTED) delay(500);
  Serial.println(WiFi.localIP());   // 手机访问这个 IP

  server.on("/",    handleRoot);    // 首页
  server.on("/on",  handleOn);      // 开灯
  server.on("/off", handleOff);     // 关灯
  server.begin();                   // 开门营业
}

void loop() {
  server.handleClient();   // 有人访问，就在这里处理
}
```

三个新东西

`WebServer server(80)`

造一个 " 前台 " 。 80 是网页的默认端口，浏览器不用写端口号就能访问。

`server.on("/on", handleOn)`

路由表：路径 → 函数。浏览器访问 /on ，就去执行 handleOn 。这是整段代码的核心思想。

`server.handleClient()`

前台必须一直坐在那儿接待。放在 loop 里，每圈问一次 " 有人来吗 " 。

把 handleClient 换成 delay(1000) 会怎样？手机点开灯，要等 1 秒才有反应，页面还经常空白——这就是 " 阻塞 " 的代价。

代码 13 （下）：三个处理函数，各管一个页面

```
void handleRoot() {
  String html = "<meta charset='utf-8'>"
                "<h2>ESP32 控制台 </h2>"
                "<a href='/on'> 开灯 </a> | <a href='/off'> 关灯 </a>";
  server.send(200, "text/html; charset=utf-8", html);
}

void handleOn() {
  digitalWrite(LED, HIGH);
  server.send(200, "text/plain; charset=utf-8", " 灯已开 ");
}

void handleOff() {
  digitalWrite(LED, LOW);
  server.send(200, "text/plain; charset=utf-8", " 灯已关 ");
}
```

send() 的三个参数

`200`	状态码。告诉浏览器 " 这次成功了 "
`"text/html"`	内容类型。网页写 html ，纯文字写 text/plain
`html / " 灯已开 "`	真正返回给浏览器的内容

一定要写 **charset=utf-8** ，否则中文在手机上全是乱码方块。

示例里的 HTML 只有三行，已经够用了：一个标题、两个链接。想好看点，就把这段换成完整的 HTML + CSS ， ESP32 完全放得下。

用法：串口打印出 IP （如 192.168.1.23 ）→ 手机连同一个 WiFi → 浏览器输入这个 IP 。

网页能开灯了，答完这 4 题再动手改

1. 选择： server.handleClient() 写在 loop 里，作用是?

A 启动服务器　　B 检查有没有人访问并处理请求　　C 关闭服务器

2. 判断：手机用 4G 流量也能打开 ESP32 的网页。

为什么：手机必须和板子在同一局域网。跨网络访问要走云端中转（第 9 章 MQTT ）。

3. 填空：网页上中文全是乱码，是因为 send 里少了 _____ 。

提示：内容类型后面要跟上字符集声明。

4. 设计：想再加一个 /status 接口，让手机能查到灯现在是开还是关，怎么写?

参考： server.on("/status", handleStatus); 函数里 server.send(200, "application/json", "{\"led\":1}") 。

参考答案　1. B　2. 错（必须同一 WiFi ）　3. charset=utf-8　4. 加一条 server.on 路由 + 一个返回 JSON 的函数

第 4 题动手做出来，你就已经会写 " 接口 " 了——所有物联网平台背后都是这种东西。

09

CHAPTER 09

MQTT： 接入物联网平台

手机在外地、板子在家里，怎么通信?

答案是找一个双方都能访问的 " 邮局 " 。

MQTT 就是一间邮局：只管投递，不管你是谁



Broker

邮局本身，一台一直在线的服务器。它不认识你的板子，只按主题分发消息。可以用公共测试服务器，也可以自己搭。

Topic

主题，像频道名，用斜杠分层： home/led 、home/temp 。谁订阅了这个主题，谁就收到消息。

QoS

送达保证等级。 0 最多一次（可能丢）， 1 至少一次（可能重复）， 2 恰好一次。控制灯用 0 就够。

为什么不用第 8 章的网页直连？因为家里路由器没有公网 IP ，外面的手机根本找不到它。 MQTT 让双方都主动去找 Broker ，问题就没了。

代码 14（上）：连上邮局，订阅自己的信箱

```
#include <WiFi.h>
#include <PubSubClient.h>

const char* TOPIC  = "wb/esp32/led01";        // 收指令
const char* STATUS = "wb/esp32/led01/status"; // 报状态
WiFiClient net;                   // 底层 TCP 连接
PubSubClient mqtt(net);           // 在它上面跑 MQTT

void setup() {
 Serial.begin(115200);
 pinMode(2, OUTPUT);

 WiFi.begin("SSID", "PASS");
 while (WiFi.status() != WL_CONNECTED) delay(500);

 mqtt.setServer("broker.emqx.io", 1883);  // 邮局地址
 mqtt.setCallback(onMessage);          // 收到消息谁来处理

 while (!mqtt.connected()) {           // 连不上就一直重试
  if (mqtt.connect("esp32-wb-01")) {    // ID 必须唯一
   Serial.println("MQTT 已连接");
   mqtt.subscribe(TOPIC);             // 订阅主题
  } else {
   delay(2000);
  }
 }
}
```

先装库，再看四行关键句

库管理器搜 **PubSubClient**（Nick O'Leary）。它只负责 MQTT 协议，网络连接交给 WiFiClient。

`PubSubClient mqtt(net)`

把 MQTT 架在 TCP 之上。就像寄信要先把信封交给快递。

`setServer(地址, 1883)`

1883 是 MQTT 的默认端口；8883 是加密版，需要证书。

`setCallback(onMessage)`

登记 " 收到消息时给谁打电话 " 。函数名传进去，不写括号。

`subscribe(TOPIC)`

订阅以后，任何人往这个主题发消息，你都会收到一份。

客户端 ID 必须全网唯一。两个人用同一个 ID，会互相把对方踢下线。

代码 14（下）：收到消息就动手，顺便报个平安

```
void onMessage(char* topic, byte* payload, unsigned int len) {
  String msg;
  for (unsigned int i = 0; i < len; i++) {
    msg += (char)payload[i];     // 逐字节拼成字符串
  }
  Serial.printf("[%s] %s\n", topic, msg.c_str());

  if (msg == "on")  digitalWrite(2, HIGH);
  if (msg == "off") digitalWrite(2, LOW);
}

void loop() {
  if (!mqtt.connected()) {      // 掉线就重连 + 重新订阅
    mqtt.connect("esp32-wb-01");
    mqtt.subscribe(TOPIC);
  }
  mqtt.loop();                  // 必须一直调用，否则收不到
  mqtt.publish(STATUS, "alive"); // 每 5 秒上报一次状态
  delay(5000);
}
```

三个容易忽略的点

`payload` 是字节数组

不是字符串，长度靠 len 参数给。所以要一个字节一个字节拼成 String 才能比较。

`mqtt.loop()` 不能省

它负责收数据、维持心跳。忘了写，连接会超时断开，消息一条也收不到。

重连后要重新订阅

Broker 不记得你订阅过什么。掉线一次，订阅关系就没了。

怎么测试

电脑装 MQTTX （或手机装 "MQTT 客户端 "），连同一个 Broker，向 wb/esp32/led01 发一条 on，板载灯就该亮。

上报用另一个主题 /status 。如果往自己订阅的主题发，你会收到自己发的消息，日志瞬间被刷满。

已经能远程控制了，来答这 4 题

1. 选择：手机在外地，凭什么能和家里的 ESP32 通信?

A 家里路由器有公网 IP　　B 双方都主动连到同一个 Broker　　C 靠蓝牙

2. 判断：板子发布消息用 home/led，自己又订阅了 home/led，不会出问题。

为什么：会收到自己发的消息。如果里面还有 " 收到就转发 " 的逻辑，会形成死循环，把 Broker 刷爆。

3. 填空：代码里忘了写 mqtt._____()，结果一条消息也收不到，连接还老是断。

提示：它负责收数据和维持心跳，必须放在 loop 里反复调用。

4. 解释：为什么订阅之后，必须要在重连时重新 subscribe 一次?

答：订阅关系存在 Broker 的连接会话里，断线就没了。 Broker 不会替你记着。

参考答案　1. B　2. 错（会收到自己的消息）　3. loop　4. 订阅关系随连接消失，重连后要重新订阅

第 3 、 4 题是 MQTT 项目最常见的两个 " 玄学故障 "，背下来能省好几天。

10

CHAPTER 10

掉电不丢：存储与配置

拔掉电源， WiFi 密码就忘了？

这一章教你把设置写进 Flash ， 下次开机自动读回来。

代码 15：把 WiFi 密码存进 Flash，断电也不丢

```
#include <Preferences.h>
Preferences prefs;              // 一个存取管理器

void saveWifi(const char* ssid, const char* pass) {
  prefs.begin("wifi", false);     // false = 读写模式
  prefs.putString("ssid", ssid);
  prefs.putString("pass", pass);
  prefs.end();                    // 用完必须关
}

void loadWifi() {
  prefs.begin("wifi", true);      // true = 只读，不创建
  String ssid = prefs.getString("ssid", "");  // 第二个参数是默认值
  String pass = prefs.getString("pass", "");
  prefs.end();
  Serial.printf("读到 ssid=%s，长度 %d\n", ssid.c_str(), pass.length());
}

void setup() {
  Serial.begin(115200);
  saveWifi("MyHome", "12345678");  // 第一次写入
  loadWifi();                     // 再读回来看看
}

void loop() {}
```

它存在哪儿：NVS

Preferences 底层是 ESP32 的 NVS（非易失存储），写在 4MB Flash 的一个小分区里。掉电不丢，擦写次数十万次量级。

`begin("wifi", false)`

"wifi" 是命名空间，相当于一个文件夹。不同项目用不同名字，互不干扰。

`putString / getString`

支持 String、int、bool、float、bytes。getString 的第二个参数是"没存过时返回什么"。

`prefs.end()`

每次 open 都会占用资源，用完必须关。忘了关，下一次 begin 会失败。

典型用法：第一次开机开热点让用户填 WiFi，存进 NVS；以后每次开机直接读出来连。

配置能存住了，回答这 4 题

1. 选择： prefs.begin("cfg", true) 里的 true 表示?

A 只读模式，命名空间不存在也不创建　　B 读写模式　　C 清空数据

2. 判断：代码里写完就拔电，数据一定能保住。

为什么： put 之后要 end() 才会真正落盘。写完立刻断电，可能还在缓存里，数据就丢了。

3. 填空： getString("ssid", " 默认值 ") 的第二个参数，作用是 _____ 。

提示：从来没存过的时候，它决定你拿到什么，避免读出空字符串导致连不上。

4. 设计：想让设备 " 第一次开机开热点配网，之后自动连 " ，整体流程怎么设计?

参考：开机读 NVS 的 ssid → 空则进 AP 模式开网页配网 → 用户提交后存 NVS 并重启 → 下次读到就直接连。

参考答案　1. A　2. 错（必须 end() 落盘）　3. 没存过时返回的默认值　4. 读 NVS → 空则开热点配网 → 存 → 重启

第 4 题是真实产品的标准做法，能画出来，说明你已经在用工程师的思路想问题了。

11

CHAPTER 11

同时做多件事

又要闪灯、又要读传感器、还要响应网页——

一个 loop 怎么忙得过来？两种方法，从简到难。

代码 16：看表干活，谁也不用等谁

```
#define LED 2
unsigned long tLed = 0;     // 灯上次翻转的时刻
unsigned long tPrint = 0;   // 上次打印的时刻

void setup() {
  Serial.begin(115200);
  pinMode(LED, OUTPUT);
}

void loop() {
  unsigned long now = millis();   // 每秒看表几十万次

  if (now - tLed >= 500) {        // 够 500ms 就闪一下
    tLed = now;                   // 记下这次的时间
    digitalWrite(LED, !digitalRead(LED));
  }

  if (now - tPrint >= 2000) {     // 够 2s 就打印一次
    tPrint = now;
    Serial.printf("运行了 %lu 毫秒\n", now);
  }

  // 这里还能继续加：读传感器、刷新屏幕……
  // 谁也不挡谁，因为没有任何 delay
}
```

核心就一句：不是睡觉，是看表

delay 是 " 我要睡 1 秒，什么都别叫我 " ； **millis** 是 " 我每秒看几十万次表，时间到了就干 " 。

✗ delay(1000)：这 1 秒里网页点不动、按键没反应

✓ millis 判断：所有任务每圈都被检查一遍

两个必须记住的细节

① 用 now - tLed >= 500 ，不要写 now - tLed == 500 。程序跑得快，几乎不可能正好等于。

② 每个任务配一个自己的时间戳变量，别共用，否则互相覆盖。

millis 约 49.7 天归零。用减法比较，归零那一刻也不会出错——这是 C 语言无符号数运算的特性。

代码 17：两块核，各干各的

```
void blinkTask(void* pv) {
  pinMode(2, OUTPUT);
  for (;;) {                  // 任务不能返回，必须死循环
    digitalWrite(2, !digitalRead(2));
    vTaskDelay(500 / portTICK_PERIOD_MS);  // 让出 CPU
  }
}

void printTask(void* pv) {
  for (;;) {
    Serial.printf("任务在跑：%lu\n", millis());
    vTaskDelay(1000 / portTICK_PERIOD_MS);
  }
}

void setup() {
  Serial.begin(115200);
  // 函数、名字、栈字节、参数、优先级、句柄、绑哪个核
  xTaskCreatePinnedToCore(blinkTask, "blink", 2048, NULL, 1, NULL, 1);
  xTaskCreatePinnedToCore(printTask, "print", 2048, NULL, 1, NULL, 0);
}

void loop() {
  Serial.println("主循环还在跑");
  delay(3000);
}
```

ESP32 本来就是双核

Core 0 默认跑 WiFi 与蓝牙协议栈，Core 1 跑你的 setup/loop。所以哪怕什么都不写，WiFi 也在"偷偷"占用一个核。

`xTaskCreatePinnedToCore`

创建一个任务并指定绑在哪个核上。最后一个参数 1 就是 Core 1。

`2048`

栈空间（字节）。里面要放局部变量就往上加，一般 2048–4096 够用。

`vTaskDelay(...)`

任务里必须有它。不写就等于死循环霸占 CPU，几秒后看门狗直接把板子重启。

什么时候才用？任务里有几百毫秒以上的耗时操作（刷屏、读慢速传感器），或者需要严格节奏。普通项目用 millis 就够了。

程序不再 " 卡住 " 了，答这 4 题

1. 选择：网页点了没反应，同时板载灯闪得很规律，最可能的原因是?

A 灯坏了　　B loop 里有 delay 把 handleClient 卡住了　　C WiFi 断线

2. 判断：判断时间到没到，写成 if (now - tLed == 500) 更准确。

为什么：几乎不可能正好等于 500，程序跑一圈可能就跳过这个点，要写 >= 。

3. 填空： FreeRTOS 任务函数里必须调用 _____，否则会被看门狗复位。

提示：它的作用是 " 让出 CPU 一小会儿 "，和 delay 类似但更轻量。

4. 解释： millis 大约 49.7 天会归零，为什么用减法比较就不会出错?

思路：无符号数相减，归零瞬间 now 很小、 tLed 很大，结果溢出回绕成正确的小正数，仍然成立。

参考答案　1. B　2. 错（要写 >= ）　3. vTaskDelay　4. 无符号减法溢出回绕，结果依然正确

第 1 题是所有 " 网页卡、按键不灵 " 的总根源，遇到先搜代码里的 delay 。

12

CHAPTER 12

综合项目：桌面气象站

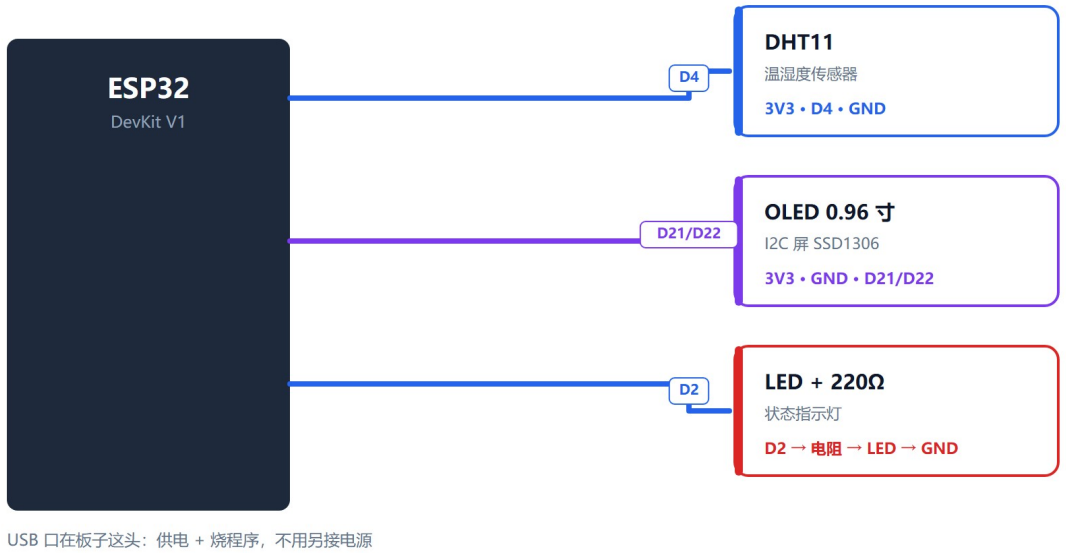
把前 11 章全用上：传感器、屏幕、 WiFi 、网页。

这一章做完，你就有一个能拿出手的作品了。

系统接线：三件设备挂在同一块板子上

接线图 · 桌面气象站

DHT11 读温湿度、OLED 显示、LED 做状态灯——三件都挂在一块板子上



USB 口在板子这头：供电 + 烧程序，不用另接电源

这张图 = 第 5 章两张接线图合起来（DHT11 + OLED），再加一颗 LED；三者的 3V3 和 GND 都并到板子上。

WIRING

先分开调通，再合起来

- D4 → DHT11 的 DATA
- D21 / D22 → OLED 的 SDA / SCL
- D2 → 220Ω → LED → GND
- 三个模块的 3V3 、 GND 都并到板子的同一组脚上

合起来不工作，九成是电源脚接错，或者 D21/D22 接反了——先拔掉一块一块试。

先定需求，再画结构，最后才写代码

五条需求，一条都不能少

01 每 2 秒采集一次温湿度

02 OLED 上实时显示，字要大

03 连上家里 WiFi，串口打印 IP

04 手机浏览器能查到当前数值

05 读取失败时不能死机、不能刷屏乱码

第 05 条最容易被忽略，却决定这个作品能不能连续跑一周。



代码 18 （上）：把四个库摆好，一次初始化完

```
#include <WiFi.h>
#include <WebServer.h>
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <DHT.h>

DHT dht(4, DHT11);                    // 传感器
Adafruit_SSD1306 oled(128, 64, &Wire, -1);  // 屏幕
WebServer server(80);                 // 网页
float temp = 0, humi = 0;             // 全局变量

void setup() {
  Serial.begin(115200);
  Wire.begin(21, 22);
  oled.begin(SSD1306_SWITCHCAPVCC, 0x3C);
  dht.begin();

  WiFi.begin("SSID", "PASS");
  while (WiFi.status() != WL_CONNECTED) delay(500);
  Serial.println(WiFi.localIP());     // 手机访问这个地址

  server.on("/data", handleData);     // 注册一个 JSON 接口
  server.begin();
}
```

为什么用全局变量

temp 和 humi 在 setup 之外声明，是为了让采集函数和网页函数共用同一份数据——谁需要谁取，不用来回传参。

`DHT dht(4, DHT11)`

引脚 4，型号 DHT11。换 DHT22 只改这里。

`Wire.begin(21, 22)`

先把 I2C 打开，OLED 才有得用。顺序不能反。

`server.on("/data", ...)`

给网页留一个接口。打开这个地址，返回一段 JSON。

初始化顺序建议：串口 → 总线 → 传感器 → 屏幕 → WiFi → 服务器。哪一步失败，串口就能立刻告诉你。

代码 18 （下）：主循环只做三件事

```
void handleData() {              // 手机访问 /data 时执行
  char buf[96];
  snprintf(buf, sizeof(buf), "{\"t\":%.1f,\"h\":%.1f}", temp, humi);
  server.send(200, "application/json", buf);
}

void loop() {
  server.handleClient();         // ① 每圈都接待网页访客

  static unsigned long t0 = 0;
  if (millis() - t0 >= 2000) {   // ② 每 2 秒采一次
    t0 = millis();
    temp = dht.readTemperature();
    humi = dht.readHumidity();

    if (!isnan(temp) && !isnan(humi)) {  // ③ 数据有效才显示
      oled.clearDisplay();
      oled.setTextSize(2);
      oled.setCursor(0, 0);
      oled.printf("%.1f C\n", temp);
      oled.printf("%.1f %%\n", humi);
      oled.display();
      Serial.printf("t=%.1f h=%.1f\n", temp, humi);
    }
  }
}
```

三件事，顺序有讲究

① 先接待网页

放在最前面，网页响应才快。放后面就会被传感器拖慢。

② 再看表采集

millis 判断，全程没有 delay，网页随时能进来。

③ 数据有效才刷屏

isnan 挡住错误值。不挡的话屏幕会闪出 nan，看着像坏了。

static 的作用：让 t0 只初始化一次，之后跨循环保留上次的值。不加 static，它每圈都变回 0，永远采不到数据。

手机打开 http:// 板子 IP/data，会看到 {"t":18.0,"h":56.0} —— 这就是一个真正的物联网接口。

上线前，按这 12 条挨个打勾

- [ ] 开发板选的是 ESP32 Dev Module
- [ ] 所有模块 VCC 接 3V3，不是 5V
- [ ] 所有 GND 都连在一起（共地）
- [ ] I2C 的 SDA/SCL 没有接反
- [ ] 裸传感器 DATA 加了上拉电阻
- [ ] WiFi 是 2.4G，密码没写错
- [ ] 串口监视器波特率 115200
- [ ] 手机和板子在同一个 WiFi
- [ ] 网页地址是 http:// 开头
- [ ] loop 里没有超过 100ms 的 delay
- [ ] 读到 nan 时有兜底，不会刷屏
- [ ] 连续通电 24 小时不重启

第 12 条是一道分水岭

能连续跑满 24 小时的，才叫 " 作品 " 。跑两小时就重启的，多半是内存泄漏、看门狗超时或供电不稳——打开串口，看重启前最后一行日志。

最后一次小考，都是真问题

1. 选择：网页打开很慢、要等 2 秒才出来，最可能是?

A WiFi 信号差　　B loop 里先采了传感器再 handleClient　　C 屏幕分辨率太高

2. 判断： loop 里的 static unsigned long t0 可以改成普通局部变量。

为什么：普通局部变量每圈都重新初始化为 0， millis() 永远减不出 2000，采集一次也不会发生。

3. 填空：想让手机直接拿到数据，网页接口返回的类型是 _____ 。

提示：不是 text/html，而是 application/json 。

4. 设计：现在只有温湿度，如果再加一个 " 温度超过 30 度就让灯快闪报警 "，怎么改?

参考：在采集到有效数据后加 if (temp > 30) 分支，用一个标志位控制闪烁间隔（250ms），低于阈值恢复 1000ms 。

参考答案　1. B　2. 错（会丢失上次时间）　3. application/json　4. 加阈值判断 + 用变量控制闪烁间隔

第 4 题做出来，这个气象站就从 " 练习 " 变成 " 产品 " 了。

结课大考 · A 卷

选择题：10 题，每题 4 分，共 40 分

1. ESP32 引脚的高电平电压是?

A 5V　　B 3.3V　　C 1.8V

2. 联网项目里模拟传感器必须接哪组引脚?

A GPIO32–39　　B GPIO0–15　　C 任意

3. delay(1000) 执行期间，芯片在做什么?

A 处理网页请求　　B 原地空等，什么都不干　　C 休眠

4. I2C 的两根信号线是?

A TX/RX　　B SDA/SCL　　C MOSI/MISO

5. 上传程序一直超时，最先试哪一招?

A 换板子　　B 按住 BOOT 再点上传　　C 重装系统

6. map(512, 0, 4095, 0, 100) 的结果约为?

A 12　　B 25　　C 50

7. OLED 写完不调用 display() 会怎样?

A 屏幕全黑　　B 正常显示　　C 花屏

8. 掉电后还要保留设置，该用什么?

A delay()　　B Preferences （NVS）　　C 全局变量

9. 手机在外地用 4G 控制家里的板子，靠的是?

A 网页直连　　B MQTT 经 Broker 中转　　C 蓝牙

10. loop 里同时要闪灯和响应网页，正确做法是?

A 都用 delay　　B 用 millis 判断时间　　C 把网页关掉

限时 15 分钟。答案在第 84 页。答对 8 题以上，说明概念已经扎实；错 3 题以上，回到对应章节重看那一页的表格。

结课大考 · B 卷

读代码 10 分 · 改错 10 分 · 设计 40 分

一、读代码：这盏灯亮多久？

```
void loop() {
  digitalWrite(2, HIGH);
  delay(300);
  digitalWrite(2, LOW);
  delay(700);
}
```

二、改错：这段按键代码有两处错

```
#define BTN 0
void setup() {
  pinMode(BTN, INPUT);
  Serial.begin(9600);
}
void loop() {
  if (digitalRead(BTN) == LOW)
    Serial.println(" 按下 ");
}
```

三、设计题（40 分）

做一个 " 按键 + 网页 + 云端 " 三合一的设备：按一下按键切换灯的状态，手机网页能看到当前状态，同时通过 MQTT 上报给云端。

要求写出三部分：

① 引脚分配表（每个引脚接什么、为什么选它）

② 网页路由设计（有哪些路径、各返回什么）

③ 主循环结构（用伪代码写清先后顺序，说明为什么这样排）

评分看四点：引脚选得对不对（10 分）、状态是否只在 " 按下瞬间 " 翻转（10 分）、网页与 MQTT 是否共用同一份状态（10 分）、主循环里有没有阻塞（10 分）。

这题的答案，其实就是第 12 章气象站的变形。能独立写出来，这门课就算真的学完了。

答案与评分标准

A 卷（每题 4 分）

1. **B**	2. **A**	3. **B**	4. **B**	5. **B**
6. **A**	7. **A**	8. **B**	9. **B**	10. **B**

第 6 题：512/4095 × 100 ≈ 12.5，取 12。

B 卷一：读代码（10 分）

亮 300 毫秒、灭 700 毫秒，周期 1 秒，占空比 30%。

得分点：说出 "亮 300 / 灭 700" 给 6 分，说出占空比 30% 再给 4 分。

B 卷二：改错（10 分）

① INPUT 要改成 INPUT_PULLUP，否则引脚浮空乱触发；

② 串口 9600 和监视器 115200 不一致，会看到乱码。每处 5 分。

B 卷三：设计题（40 分）参考答案

① **引脚：**按键 GPIO0（板上自带，内部上拉）；LED GPIO2（板载，免接线）；不占 34–39，留给未来的模拟传感器。

② **路由：**/ 返回控制网页；/on 开灯返回 text；/off 关灯；/status 返回 {"led":1}。

③ **主循环：**先 handleClient → 再 mqtt.loop → 再读按键（millis 判 30ms 稳定）→ 状态变化才翻转并上报。全程无 delay。

扣分点：用 delay 消抖（-10）、在按键回调里直接改灯却不更新全局状态（-10）、网页和 MQTT 各存一份状态导致不一致（-10）。

60 分及格，80 分以上可以去接真实的物联网小项目了。

结课

接下来 90 天，把 " 学过 " 变成 " 会做 "

第 1–30 天

把手练熟

· 12 章的例子全部亲手敲一遍，不许复制

· 每章的 " 改一改 " 至少做两个

· 做完 12 次小考，错题记在本子上

第 31–60 天

做一个自己的东西

· 选一个你真正想要的：鱼缸水温、植物浇水、门口感应灯

· 先写需求，再画结构，最后写代码

· 把 WiFi 密码存进 NVS，做成能独立运行的样子

第 61–90 天

往外走一步

· 加 OTA 远程升级，或做低功耗电池版

· 换 ESP32-S3 上 LVGL 界面 / 摄像头

· 把作品发到立创开源或 B 站，接受别人的提问

遇到问题去哪儿查

官方文档 docs.espressif.com （最准） · Arduino-ESP32 的 GitHub 示例（最全） · 把报错原文粘进搜索框（最快）

最后一句忠告：**先让它跑起来，再让它变好。**卡住超过两小时，就换一种实现方式绕过去。

COURSE COMPLETE

从点亮一颗灯，到让世界看见你的数据

把这 12 章做完的人，已经超过大多数买过开发板的人。

剩下的路，靠你自己动手了。

作者：老张 · 邮箱：xlgxxc@126.com

ESP32 零基础入门教程 · 配套硬件 ESP32 DevKit V1 · 2026